

CONTEXTGRAPH: MEMORY FOR INCOMPLETE REPAIRS

Anonymous authors

Paper under double-blind review

ABSTRACT

A repair can satisfy the reported issue while retaining a fault that past experience already explains. We study candidate-conditioned repair memory: use a historical repair to identify a behavioral requirement missing from an initial candidate, then revise that candidate while preserving its existing fix. An executable source relation makes this omission observable on the patched target. On a Django development task, an initial repair and an additional issue-based review both pass official verification but discard query parameters during redirection. A historical repair enables a revision that preserves both the required deployment prefix and the omitted query parameters, including a joint input withheld from the writers. Supplying the historical patch and supplying an executed source check produce identical revisions. A separate development task yields four candidates that already satisfy the source condition. Together, these observations locate the useful role of memory in requirements a candidate still violates, while providing no evidence that executing the relation improves revision over supplying the relevant patch.

1 INTRODUCTION

The issue description tells a coding agent what is currently reported as broken. It need not enumerate every behavior a complete repair should preserve. A retrieved experience is useful when it supplies a missing requirement that changes the candidate repair. Similarity to the issue, or a new observation about the program, is insufficient if the candidate already handles that condition.

Consider an administrative redirect that must preserve a deployment prefix. Changing its destination from `request.path_info` to `request.path` fixes that reported fault. However, appending a slash to the resulting path still discards query parameters. A different repair of the same API records this omitted behavior and replaces path concatenation with `request.get_full_path(force_append_slash=True)`.

We investigate a procedure organized around this residual defect. First construct a candidate from the current issue. Then assess a relevant historical condition on the patched program and revise when that condition exposes an omission compatible with the current task. This makes the proposed repair, rather than issue relevance alone, the object of the memory decision. Our development evidence distinguishes this useful omission from a historical condition already covered by ordinary repairs.

2 MEMORY CONDITIONED ON THE CANDIDATE REPAIR

Recover the historical requirement. A source record contains a buggy program, its repair, and the repaired program. A writer receives the patch and relevant repaired code and generates a short executable check of the repair’s observable effect. We execute it in both source versions. The check must run normally, hold after the repair on the supplied legal inputs, and change from false to true on at least one input. The resulting memory records the relation and the behavior demonstrating its relevance to the source repair.

Look for a remaining violation. Let p be the initial candidate for target program T , and let r be a source relation that can be instantiated on that target. We evaluate r on $T \oplus p$, the program

Table 1: Django 14404 development repair. Every row passes official verification: 2 fail-to-pass and 323 pass-to-pass tests. Behavioral checks distinguish the omitted requirement.

Candidate	Prefix	Query	Prefix + query
Initial repair	Pass	Fail	Fail
Issue-based revision	Pass	Fail	Fail
Historical patch revision	Pass	Pass	Pass
Executed relation revision	Pass	Pass	Pass

after applying the candidate. In the experiments below, we select a source repair of the same API and execute its generated check unchanged on the target candidate. A normal execution with a false result identifies a possible omission; a true result means that this relation provides no reason to revise the candidate. The target’s required behavior determines whether the source condition is relevant. Execution errors provide neither observation.

This distinction matters when every candidate receives the same source-check result. If all satisfy the relation, it supplies no selection value. If all violate a relevant relation, ranking the same candidates cannot recover the missing behavior; the useful operation is to generate a corrective edit. The procedure therefore uses a violated source condition to request a revision and instructs the writer to preserve the behavior the initial candidate already fixes.

Revise using the experience. The revision writer receives the issue, initial implementation, candidate, and observations of the reported fault after the candidate is applied. Historical information can be supplied as the source issue and patch, or as the source relation, its before/after observations, and the check’s observed failure on the candidate. The writer makes one revision call and returns a complete revised method. Each resulting patch is immediately evaluated by the official SWE-bench verifier (Jimenez et al., 2024); the behavioral checks then measure whether the reported requirement and historical condition hold together. The experiment supplies a selected source repair so that this comparison isolates its contribution to revision.

3 EVIDENCE: RECOVERING AN OMITTED REQUIREMENT

We use the training side of a SWE-bench Verified split with 280 source tasks, 70 development tasks, and a separate 150-task evaluation partition unused by these experiments. The principal pair is source Django 16612 and development target Django 14404. The target issue asks for support of `FORCE_SCRIPT_NAME` in administrative redirects. The source condition concerns query preservation in the same method.

An ordinary repair leaves the historical fault. DeepSeek V4 Pro generates one initial repair from the issue and relevant initial code, including the available request-path APIs. The writer may replace `AdminSite.catch_all_view()` while preserving its signature and decorator. Its repair keeps URL resolution based on `path_info` and constructs the redirect from `request.path`. Official verification passes, and an issue-based check confirms that the deployment prefix is retained. The unchanged source program executes normally on this candidate and observes that both query strings are discarded.

Memory changes the revision. Starting from that same candidate, three independent revision calls receive either the issue-based evidence alone, the historical issue and patch, or the executed source relation. All use the same model, temperature 0.7, and one call with a 16,384-token output allowance. The issue-based review retains the initial patch. Both memory conditions replace path concatenation with `get_full_path(force_append_slash=True)`, producing byte-identical patches. Those revisions retain the deployment prefix and preserve the query parameters (Table 1).

The revision preserves the requirements together. An independently prepared joint check combines the deployment prefix with repeated query parameters and percent-encoded values:

keep=a%2Fb&keep=c%20d. This input is withheld from all writers and executed after their revisions and official verifications complete. Both memory revisions preserve the exact prefix and query string in the redirect; the initial and issue-reviewed candidates lose the query string.

A covered condition contributes no corrective signal. Source Django 16493 records that serializing a `FileField` should preserve its original storage factory even when the factory returns `default_storage`. On development target Django 13343, four independent source-blind candidates produce the same AST repair and all satisfy that relation. Each also passes official verification. The source condition therefore supplies no additional candidate-selection value in that pool.

The positive case establishes a concrete corrective contribution from historical information, not an improvement in official resolved rate or an advantage of executable delivery over the patch.

4 RELATED WORK

ExpeL distills reusable insights from experience, and Getafix learns applicable edit patterns from human repairs (Zhao et al., 2024; Bader et al., 2019). ExpeRepair uses historical test demonstrations and reflective insights to generate reproduction and boundary tests (Mu et al., 2026). MIRA constructs repair memory from SQL corrections and uses database evidence to activate and combine applicable repair units (Liu et al., 2026). These works already establish repair reuse, historical test guidance, and execution-based memory activation. Our empirical focus is the residual requirement: whether historical information changes a candidate that already fixes the reported fault, and whether the resulting revision preserves the old and new requirements together.

AI USE STATEMENT

Generative AI assisted literature review, method development, implementation, experiments, analysis, and writing.

REFERENCES

- Johannes Bader, Andrew Scott, Michael Pradel, and Satish Chandra. Getafix: Learning to fix bugs automatically, 2019. URL <https://arxiv.org/abs/1902.06111>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Yining Liu, Chenyu Yang, Boyan Li, Rui Mao, and Yuyu Luo. MIRA: Evidence-verified repair memory for text-to-sql correction, 2026. URL <https://arxiv.org/abs/2608.06950v1>.
- Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. ExpeRepair: Dual-memory enhanced LLM-based repository-level program repair, 2026. URL <https://arxiv.org/abs/2506.10484v2>. Version 2; accepted at FSE 2026.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, 2024. URL <https://arxiv.org/abs/2308.10144v3>.